Reinforcement Learning (DSAI 402)

Lecture 3: Markov Decision Processes

Prof. Mohamed Ghalwash
mghalwash@zewailcity.edu.eg

Zewail City University

Lecture 3, Monday 5 October 2026

Agenda

Objectives: you should be able to write down an MDP for a small problem, explain what a policy and a value function are, and use the Bellman equation to compute a value.

- Play: program a robot on a slippery grid, no vocabulary yet
- Name it: policy, return, value
- Formalize it: the MDP, the Bellman equations, optimal policies
- Apply it: the second application paper, chip floorplanning

Programming a Robot on a Slippery Grid

- Start at 🟢 and reach ⭐ to end the game on a **fully visible map**. There is no move limit.
- Pre-assign **one arrow per square** for the robot to follow. Hitting a wall keeps the robot in place.
- The floor is **slippery** so a die roll determines movement:
  - 1–4 follows the arrow, 5 or 6 turns 90° left or right.
- Points: -1 for any move, -5 for landing on 💥, and $+10$ for reaching ⭐. Your score is the sum of the points over the whole round, e.g., The move that lands on 💥 gives $-1 - 5 = -6$.

	Col 1	Col 2	Col 3	Col 4
Row 1	·	·	·	·
Row 2	🟢	💥	·	⭐
Row 3	·	·	💥	·

Watch the Robot Run

Only the squares the robot visits are shown. Squares are (row, column).

Move	Square	Arrow	Roll	The robot goes	Points	Total
1	(2,1)	→	2	→ onto the 💥	-1 - 5 = -6	-6
2	(2,2)	→	5	↑	-1	-7
3	(1,2)	→	1	→	-1	-8
4	(1,3)	→	6	↓	-1	-9
5	(2,3)	→	3	→ onto the ⭐	-1 + 10 = +9	0

The robot never asked the team anything. It read the arrow of its square, and the die decided the rest.

Write the Arrows

- Copy the grid on paper. Draw **one arrow in every square** except the ⭐. That is 11 arrows.
- That includes the 🟢 square and the 💥 squares.
- The robot cannot ask you what to do, and a slip can carry it onto any square.
- Once the dice start, the arrows cannot change.
- You have 4 minutes, as a team.

	Col 1	Col 2	Col 3	Col 4
Row 1	?	?	?	?
Row 2	🟢 ?	💥 ?	?	⭐
Row 3	?	?	💥 ?	?

Round 1: Roll the Dice

Move 1 uses the first roll, move 2 the second, and so on. All teams use the same rolls. Follow your arrows exactly. Stop when the robot lands on the ⭐.

Round 2: Same Arrows But New Dice

Keep your arrows. Start again from 🟢 with these rolls.

Debrief

- What did you give the robot before it started?
- Why was a path not enough?
- Look at the board. Did the best team of round 1 also win round 2?
- So how should we judge a set of arrows?
- Is every square worth the same to the robot?
- last week, by locking one action per cell the grid transformed into a Markov process. What did you just do?

You Already Used All of These

Dynamics ($p(s', r \mid s, a)$)

The chance of each next square and reward, given the square and the arrow.

- Game: the dice table and the points

Policy ($\pi(a \mid s)$)

A rule that picks an action in every state.

- Game: your arrows

Return (G_t)

The total reward of one run.

- Game: your total in one round

Value ($v_\pi(s)$)

The average return when you start in s and follow π.

- Game: the average total over many rounds that start on 🟢

Known model

This time you knew the dice and the map. Last week you did not.

- Weeks 3 and 4 use a known model. Weeks 5 to 7 do not.

Important

A return is what happened in one run. A value is what you expect on average. They are not the same thing.

The Best Arrows and What Each Square Is Worth

	Col 1	Col 2	Col 3	Col 4
Row 1	→ 1.9	→ 3.9	→ 6.6	↓ 8.1
Row 2	🟢 ↑ 0.1	💥 → 3.8	→ 6.7	⭐ 0
Row 3	↑ -1.5	→ -1.8	💥 → 4.5	↑ 6.7

- Total points:
  - Round 1: **4, 5, 5, 1, 2, 3, 6, 6, 6, 2, 2, 3** ⇒ -6
  - Round 2: **2, 3, 1, 6, 4** ⇒ +5
  - Average across many rounds ⇒ +0.1
- Each number is the average total from that square on, when the robot follows the arrows in the table.

How Do We Get These Averages?

We do not need 20,000 rounds. Take the square in the top right corner, where the arrow is ↓.

- Rolls 1 to 4 (4 out of 6): the robot lands on the ⭐ and gets $-1 + 10 = 9$.
- Roll 5 (1 out of 6): the move turns to →, a wall stops the robot, and it pays 1 and stays.
- Roll 6 (1 out of 6): the move turns to ←, and the robot pays 1 and lands one square to the left, which is worth 6.6.

$$v = \tfrac{4}{6}(9) + \tfrac{1}{6}(-1 + v) + \tfrac{1}{6}(-1 + 6.6) \quad \Rightarrow \quad v \approx 8.1$$

> **Tip**
> The value of a square is built from the values of its neighbors. This is the Bellman equation, and we write it properly in a few slides.

Markov Property

To predict what a move does, do you need your whole history, or just your square and the move itself?

$$P(S_{t+1}, R_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, \ldots, S_0, A_0) = P(S_{t+1}, R_{t+1} \mid S_t, A_t)$$

In the robot game the state is the square. Where the robot was before does not change what the next roll does.

- A chess position + active player + castling/en passant rights = full Markov state.
- A single video frame of a moving ball is not Markovian, because one frame does not show the direction of motion.

Important

The state is a design choice. If the agent needs something that the state leaves out, the problem is not an MDP yet. The fix is to put that part of the past into the state, e.g. DQN stacks the last 4 game frames.

From a Markov Process to an MDP

A Markov process has states and transition probabilities. An MDP adds a choice and a goal.

Piece	Symbol	Meaning
States	$\mathcal{S}$	The situations the agent can be in
Actions	$\mathcal{A}$	The choices the agent can make
Dynamics	$p(s', r \mid s, a)$	How the environment responds
Reward	r	A number that says how good the last step was
Discount	$\gamma \in [0, 1]$	How much future reward counts

The agent picks A_t. The environment picks S_{t+1} and R_{t+1} using p.

The Dynamics Function

$$p(s', r \mid s, a) = \Pr\{S_{t+1} = s',\ R_{t+1} = r \mid S_t = s,\ A_t = a\}$$

For every pair (s, a) the probabilities add up to one.

$$\sum_{s'} \sum_{r} p(s', r \mid s, a) = 1$$

Two numbers we get from p and use all the time

$$p(s' \mid s, a) = \sum_{r} p(s', r \mid s, a) \qquad r(s, a) = \sum_{r} \sum_{s'} r . p(s', r \mid s, a)$$

The first is the chance of landing in s'. The second is the reward we expect from taking a in s.

Designing the Reward

The reward hypothesis says that a goal can be thought of as maximizing the expected cumulative sum of one scalar signal.

Good habit. Reward the outcome you want, not the steps you think lead to it.

- Chess. Reward $+1$ for a win and -1 for a loss. Do not reward capturing pieces, because the agent may learn to trade pieces and still lose the game.
- A hypothetical cleaning robot that is paid for each piece of trash it collects may learn to make more trash.
- In our game, the -1 per move is what asks the robot to hurry. With 0 per move nothing pushes it to hurry, and with $+1$ per move it would be paid to wander.

> **Tip**
> The agent will find any shortcut the reward allows. The project mentor can help you spot these before training.

Return

The return is the discounted sum of rewards from time t on.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

It has a recursive form that we will use everywhere.

$$G_t = R_{t+1} + \gamma G_{t+1}$$

Episodic and Continuing Tasks

	Episodic	Continuing
Ends	Yes, in a terminal state	No
Examples	A game of Go, one patient stay in the ICU, our robot game	Elevator control, a data center cooler
Discount	$\gamma = 1$ is fine when every episode ends	Usually $\gamma < 1$

With $\gamma < 1$ and rewards that never exceed $R_{\max}$, the return stays finite.

$$G_t \leq \frac{R_{\max}}{1 - \gamma}$$

Small γ makes the agent short-sighted. A γ near one makes it far-sighted but usually harder to learn.

Policies

A policy is the rule the agent uses to pick actions.

$$\pi(a \mid s) = \Pr\{A_t = a \mid S_t = s\}$$

- A deterministic policy picks one action in each state. Your arrows were one.
- A stochastic policy gives a probability to each action.

Stochastic policies are used to explore while learning. They are also needed when the agent cannot see the full state or plays against other agents, like in rock-paper-scissors. In a fully observed MDP a deterministic policy is enough to be optimal.

The goal of reinforcement learning is to find a policy that gets the largest expected return.

Value Functions

The value of a state is the return we expect if we start there and follow π.

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right]$$

The value of an action is the return we expect if we take a now and follow π after that.

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid S_t = s,\ A_t = a \right]$$

The two are linked.

$$v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s, a)$$

The Bellman Equation for v_π

Start from the definition and use $G_t = R_{t+1} + \gamma G_{t+1}$.

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi \left[R_{t+1} + \gamma G_{t+1} \mid S_t = s \right] \\ &= \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \, v_\pi(s') \right] \end{aligned}$$

In words, the value of a state is the average, over what we might do and what the world might do, of the reward now plus the discounted value of where we land.

It gives one linear equation for each state. With n states there are n equations and n unknowns.

The Bellman Equation for q_π

$$q_\pi(s,a) = \sum_{s',r} p(s',r \mid s,a) \left[r + \gamma \sum_{a'} \pi(a' \mid s')\, q_\pi(s',a') \right]$$

Both equations say the same thing. The value of a choice is the reward we get now plus the discounted value of what comes next.

This is the idea behind dynamic programming next week, and behind temporal difference learning in week 6.

Solving for v_π by Hand

Two states A and B, with $\gamma = 0.5$ and a fixed policy.

- From A, with probability 0.5 we stay in A with reward 0, and with probability 0.5 we go to B with reward 2.
- From B we always go to A with reward 3.

$$v(A) = 0.5\,\big(0 + 0.5\,v(A)\big) + 0.5\,\big(2 + 0.5\,v(B)\big)$$

$$v(B) = 3 + 0.5\,v(A)$$

- Put $v(B)$ into the first equation.

$$0.75\,v(A) = 1 + 0.25\,\big(3 + 0.5\,v(A)\big) = 1.75 + 0.125\,v(A)$$

$$0.625\,v(A) = 1.75 \quad \Rightarrow \quad v(A) = 2.8$$

$$v(B) = 3 + 0.5 \cdot 2.8 = 4.4$$

Check by putting the numbers back.

Optimal Policies and Optimal Values

Policy π is at least as good as π' if $v_\pi(s) \geq v_{\pi'}(s)$ for every state.

The optimal value functions

$$v_*(s) = \max_\pi v_\pi(s) \qquad q_*(s,a) = \max_\pi q_\pi(s,a)$$

Three facts to keep

- An optimal policy always exists for a finite MDP.
- There can be many optimal policies, but they all share the same v_* and q_*.
- At least one optimal policy is deterministic.

The Bellman Optimality Equation

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \, v_*(s') \right]$$

$$q_*(s,a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]$$

The max makes these equations nonlinear, so we cannot solve them like the two-state example.

Once we know v_* and p, the best action in a state is the one that gives the largest value in the bracket. Once we know q_*, it is $\arg\max_a q_*(s,a)$ and we do not need p.

The next lectures are about ways to get these values, first when we know p and then when we do not.

Paper: Chip Floorplanning as an MDP (Nature 2021)

For this week, read the formulation only. The network and the training come back in the deep RL weeks.

1. What is the state, and how is it written for a neural network?
2. What is an action?
3. When does an episode end?
4. What is the reward, and when does the agent get it?
5. Is the transition deterministic?
6. What value of γ would you pick, and why?
7. Which parts of the design are inside the MDP, and which are done by other tools?
8. Could a human engineer have written the reward directly, or did it need a stand-in for the true goal?
9. What would you check before trusting a result like this?

Common Mistakes When Writing an MDP

- The state leaves out something the agent needs, so the problem is not Markov.
- The reward pays for a step instead of the goal, and the agent finds a shortcut.
- The reward arrives only at the very end, so the agent has almost nothing to learn from early on.
- The task never ends and $\gamma = 1$, so the return can be infinite.
- The action space is huge, for example every possible layout at once, when it could be one small choice at a time.
- Rewards are indexed wrongly. In our notation the action at time t leads to R_{t+1}.

Tip
Keep this list near you when you write the project proposal.

Practice Check

1. In the two-state example, what is $v(A)$ if $\gamma = 0$?
2. Why does a continuing task usually need $\gamma < 1$ while an episodic task may use $\gamma = 1$?
3. Give an example of a problem where the next observation is not Markov, and say how you would fix it.
4. Why is q_* more useful than v_* when we do not know the dynamics?

MDP
States, actions, dynamics, rewards, and a discount

Dynamics $p(s', r \mid s, a)$
How the environment responds

Policy $\pi(a \mid s)$
The rule that picks actions

Return G_t
Discounted total reward of one run

Value $v_\pi(s)$
Expected return from a state under π

Action value $q_\pi(s, a)$
Expected return after taking a in s

Bellman equation
A value written with the values of the next states

Optimal policy
The policy with the largest value in every state

Before We Meet Again

1. **Read Sutton and Barto, Chapter 3**, and start Chapter 4.
2. **Write a short response to the chip floorplanning paper**, for its formulation only.
3. **Read the sepsis paper for week 4**: Komorowski et al., Nature Medicine 24, 1716-1720 (2018).
4. **Project teams**: the proposal and the mentor agreement are due in week 4.

Next lecture: dynamic programming, where we solve the Bellman equations when the dynamics are known.

> **Tip**
> Your proposal needs a domain, a track, a mentor, and a first draft of the states, actions, and reward.

Learn More

Course Homepage